

Yes — **sets and lists are stored very differently in memory**. The key idea is:

List → stores references in a contiguous array-like structure.

Set → uses a hash table, so elements are placed according to their hash values.

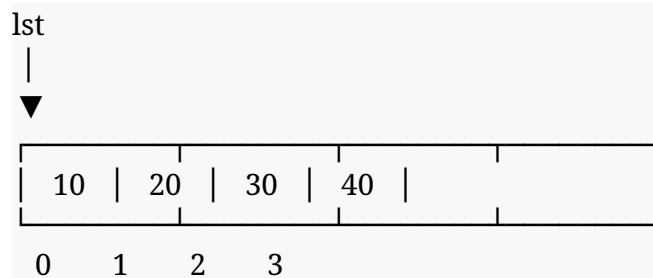
Python `list` vs `set` — Memory Structure

1. List Memory

Suppose:

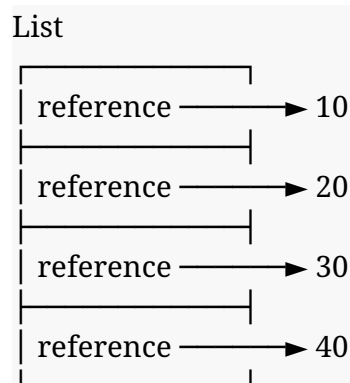
```
lst = [10, 20, 30, 40]
```

Conceptually, memory looks like:



A Python list is essentially a **dynamic array of references**.

The list's slots point to Python objects:



Important

The list maintains an **ordered sequence**.

Therefore:

```
lst[0] # 10  
lst[1] # 20  
lst[2] # 30
```

Indexing is possible.

2. Set Memory

Suppose:

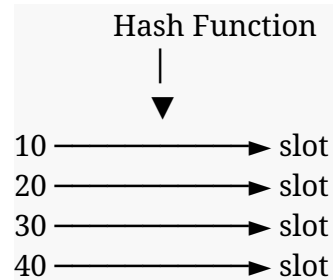
```
s = {10, 20, 30, 40}
```

A set does **not** store elements like:

10 → 20 → 30 → 40

Instead, Python's set is based on a **hash table**.

Conceptually:



Think of it like:

Hash Table

Slot 0	
Slot 1	20
Slot 2	
Slot 3	40
Slot 4	10

Slot 5	
Slot 6	30

This is conceptual, not a literal representation of Python's internal table.

The important point is that the location is determined using the element's **hash**.

3. Why Does Set Need Hashing?

When you do:

```
s = {10, 20, 30, 40}
```

```
print(30 in s)
```

Python doesn't normally scan every element one by one like:

Is 30 == 10? ❌

Is 30 == 20? ❌

Is 30 == 30? ✅

Instead, hashing helps determine where to look.

Conceptually:

30

↓

hash(30)

↓

possible table location

↓

check that location

↓

Found!

That's why set membership is **O(1) average case**.

4. List vs Set Memory

Feature	List	Set
Main structure	Dynamic array	Hash table
Stores references	✓	✓
Ordered sequence	✓	✗
Indexing	✓	✗
Hashing	✗	✓
Duplicate elements	✓	✗
Memory efficient	Generally more efficient	Generally uses more memory
Fast membership	$O(n)$ average	$O(1)$ average
Random access	$O(1)$	Not supported
Insertion	$O(1)$ amortized at end	$O(1)$ average
Deletion	$O(n)$ by value	$O(1)$ average

5. Why Set Usually Uses More Memory

Consider:

lst = [10, 20, 30, 40]

The list mainly needs an array of references:

10	20	30	40	
----	----	----	----	--

A set needs a **hash table with slots**, including unused space to maintain efficient lookup.

Conceptually:

List:

10	20	30	40	
----	----	----	----	--

Set:

	20		40	10		30			
--	----	--	----	----	--	----	--	--	--

The empty slots are important because hash tables need **space to avoid becoming too crowded**.

So:

Set trades extra memory for faster lookup.

6. Why Set Cannot Have Indexing

List:

```
lst = [10, 20, 30]
```

```
print(lst[1])
```

Output:

```
20
```

Because the list has a defined positional structure:

index

↓

0	1	2
10	20	30

A set is hash-based:

hash(10) → table location

hash(20) → table location

hash(30) → table location

There is no meaningful:

0 → first element
1 → second element
2 → third element

Therefore:

```
s[0]
```

gives:

```
TypeError
```

7. Why Duplicates Are Automatically Removed

Consider:

```
s = {10, 20, 10, 30, 20}
```

Python uses the set's hash-table machinery to maintain uniqueness.

Result:

```
{10, 20, 30}
```

Conceptually:

10 → hash → slot
20 → hash → slot
10 → same hash/equality → already exists
30 → hash → slot
20 → already exists

So a set represents **unique values**.

8. The Role of `hash()`

You can see the hash of hashable objects:

```
print(hash(10))  
print(hash("hello"))  
print(hash(3.14))
```

A set uses hashing internally.

For example:

```
s = {"apple", "banana", "mango"}
```

Conceptually:

"apple"

↓

hash()

↓

table position

"banana"

↓

hash()

↓

table position

"mango"

↓

hash()

↓

table position

This is why set elements must be **hashable**.

9. Why List Cannot Be a Set Element

This works:

```
s = {10, "hello", (1, 2)}
```

But this doesn't:

```
s = {[1, 2]}
```

because a list is **mutable** and therefore unhashable.

```
hash([1, 2])
```

produces:

TypeError: unhashable type: 'list'

But a tuple containing hashable elements can be used:

```
s = {(1, 2), (3, 4)}
```

because tuples can be hashable when their contents are hashable.

10. Very Important: Set Is NOT Just a List Without Duplicates

A common beginner misconception:

List

↓

remove duplicates

↓

Set

Not exactly.

They have fundamentally different data structures.

List

Dynamic Array

↓

position/index based

Set

Hash Table

↓

hash based

That's why their behavior is different.

11. Real-Life Analogy

List = Cinema Seats

Seat 1 → Abhi
Seat 2 → Rahul
Seat 3 → Aman
Seat 4 → Rohit

Position matters.

You can say:

“Who is sitting at seat 3?”

Set = Guest Entry System

You only care:

“Is Rahul already inside?”

Rahul → hash → lookup

You don't care whether Rahul is “person number 3”.

The main purpose is **fast membership** + **uniqueness**.

12. Memory + Performance Connection

This is the most important relationship to remember:

LIST

- Dynamic array
- Ordered
- Index based
- Membership → $O(n)$

SET

- Hash table
- No positional indexing
- Unique elements
- Membership → $O(1)$ average

The trade-off

	MEMORY	SPEED
List	↓	↓
Set	↑	↑

More accurately:

Set generally consumes more memory, but gives much faster average membership testing.

13. Interview Notes

List

arr = [10, 20, 30]

- Dynamic array
- Ordered
- Allows duplicates
- Indexing supported
- Slicing supported
- Average x in list → **$O(n)$**
- Random access → **$O(1)$**

Set

s = {10, 20, 30}

- Hash-table based
- Unique elements
- No positional indexing

- No slicing
 - Elements must be hashable
 - Average x in set → **$O(1)$**
 - Uses more memory than a list in typical cases
-

One-Line Memory Trick

LIST = “Where is it?” → position/index

SET = “Does it exist?” → hash lookup

And that’s basically the reason **list** and **set** exist as separate data structures in Python.